

Space Invaders

Smart Autonomous Car: GPS + INS + nRF24L01 + Kalman Filter

Observation

Dr. Ahmed Aboulftouh

Team Members & Supervision

Supervision

Role	Name	Department
Project Supervisor	Dr. Ahmed Aboulftouh	Aero Space Engineering and Assistant Professor Of Space Navigation

Team — 10 Members / 5 Sub-Teams

Team Lead:

Karim Ahmed — Overall team lead, EKF algorithm lead, nRF24L01 RF protocol design.

Sub-Team A — Hardware & Structure

Member	Role	Responsibilities
Nour El-din Mahmoud	Integration Lead & Line Following Lead	Full-system integration of all modules; line-following lead; final hardware assembly and validation.
Yara El-fayomi	Structure Team	Chassis design, mechanical assembly, motor mounting, physical layout of components.
Wgdan Mohammed	Structure Team	Chassis fabrication, power distribution board, L298N and battery wiring.

Sub-Team B — Line Following

Member	Role	Responsibilities
Adam Mohammed	Line Following	3x IR sensor wiring & calibration; PID line-follow code; track testing on curves and intersections.

Sub-Team C — Obstacle Avoidance

Member	Role	Responsibilities
Rahma Ahmed	Obstacle Avoidance & Tasks Manager	HC-SR04 wiring; avoidance maneuver code; speed/distance threshold calibration; overall task coordination.
Hager Khalaf	Obstacle & MATLAB Code	Obstacle testing; speed/distance tests; MATLAB dashboard development and serial parsing.

Sub-Team D — GPS & IMU

Member	Role	Responsibilities
Rania Mokhles	GPS INS Connections & Code	NEO-6M GPS wiring, TinyGPSPlus integration, coordinate conversion, GPS-INS sensor fusion connections.
Aya Saeed	MATLAB Code & GPS INS Connections	MPU-6050 gyro bias calibration; accelerometer noise analysis; MATLAB Kalman filter script; GPS-INS connections.

Sub-Team E — EKF, RF & MATLAB

Member	Role	Responsibilities
Karim Ahmed	Team Lead & EKF Lead	Extended Kalman Filter algorithm design; team coordination; nRF24L01 TX/RX protocol; 28-byte struct definition.

Abdulrahman Mostafa	EKF	Arduino EKF implementation; predict/update cycle; covariance tuning; RF packet transmission code.
Aya Saeed	MATLAB Ground Station	MATLAB Kalman filter script; 8-subplot live dashboard; serial port parsing and data validation.
Hager Khalaf	MATLAB Ground Station	MATLAB dashboard subplot design; obstacle-related telemetry visualisation; co-development of serial parser.

Complete System Concept:

Car (Arduino Nano) reads GPS lat/lon/alt, IMU ax/ay/gz, and EKF heading every 100 ms. This data is packed into a 28-byte struct and transmitted wirelessly via nRF24L01 (2.4 GHz). A second Arduino at the ground station receives the packet and forwards it as CSV over USB Serial to MATLAB. MATLAB runs a live 3-state Kalman Filter and updates an 8-subplot real-time dashboard.

1. Complete System Architecture

Data Flow — End to End

#	Stage	Device	Data Transmitted	Rate
1	Sensor Read	Arduino Nano (Car)	GPS: lat,lon,alt IMU: ax,ay,gz EKF: heading	10 Hz
2	Onboard EKF	Arduino Nano (Car)	5-state: [x,y,heading,vx,vy] — used for navigation	50 Hz
3	Pack RF Struct	Arduino Nano (Car)	struct DataPacket {lat,lon,alt,ax,ay,gz,heading}	10 Hz
4	RF Transmit	nRF24L01 TX	2.4 GHz wireless, 32-byte packet, ~100 m range	10 Hz
5	RF Receive	nRF24L01 RX (Ground Arduino)	Receive struct, forward as CSV over USB	10 Hz
6	Forward to PC	Arduino Nano (Ground)	CSV: ax,alt,gz,lat,lon,heading over Serial 115200	10 Hz
7	MATLAB Receive	MATLAB serialport()	Parse CSV into 6 variables	10 Hz
8	Kalman Filter	MATLAB	3-state KF predict+update — x[1,2,3], P, innov	10 Hz
9	Live Dashboard	MATLAB figure	8 animatedline subplots refresh via drawnow	~20 fps

RF Packet Structure — nRF24L01 (28 bytes)

```
// DATA PACKET STRUCT — Car side (transmitter)
struct DataPacket {
    float lat;           // GPS latitude   (degrees)    4 bytes
    float lon;           // GPS longitude  (degrees)    4 bytes
    float alt;           // GPS altitude   (meters)     4 bytes
    float ax;            // IMU accel X    (m/s^2)     4 bytes
    float ay;            // IMU accel Y    (m/s^2)     4 bytes
    float gz;            // IMU gyro Z     (deg/s)      4 bytes
    float heading;       // EKF heading    (degrees)    4 bytes
}; // Total: 7 x 4 = 28 bytes

DataPacket pkt;
radio.write(&pkt, sizeof(pkt));
```

nRF24L01 Wiring — Car (Transmitter)

nRF24L01 Pin	Arduino Nano Pin	Wire Color	Critical Notes
VCC	3.3V	Red	3.3V ONLY — 5V destroys module permanently
GND	GND	Black	Common ground with all other modules
CE	D7	Orange	Chip Enable — HIGH to transmit/receive
CSN	D8	Yellow	Chip Select — LOW during SPI transaction
SCK	D13	Blue	SPI Clock
MOSI	D11	Green	SPI Master Out Slave In
MISO	D12	Purple	SPI Master In Slave Out
IRQ	Not connected	—	Optional interrupt on TX/RX event

IMPORTANT — Add Decoupling Capacitors:

Place a 10 uF electrolytic capacitor AND a 100 nF ceramic capacitor between the nRF24L01 VCC and GND pins, as close to the module as possible. Without these, motor noise will corrupt the RF signal causing frequent packet drops and unreliable readings in MATLAB.

2. Arduino Code — Car (Transmitter)

Libraries Required

Library	Install Via	Purpose
RF24	Library Manager (TMRh20)	nRF24L01 driver — SPI communication
TinyGPSPlus	Library Manager (Mikal Hart)	Parse NMEA sentences from NEO-6M GPS
MPU6050	Library Manager (Electronic Cats)	Read 6-axis IMU over I2C
Wire	Arduino Built-in	I2C bus for MPU-6050
SoftwareSerial	Arduino Built-in	UART emulation for GPS

Complete Transmitter Code

```
#include <SPI.h>
#include <RF24.h>
#include <Wire.h>
#include <MPU6050.h>
#include <TinyGPSPlus.h>
#include <SoftwareSerial.h>

RF24          radio(7, 8);           // CE=D7, CSN=D8
const byte    addr[6] = "CAR01";
MPU6050       imu;
TinyGPSPlus   gps;
SoftwareSerial gpsSerial(11, 12);    // GPS RX=D11

struct DataPacket { float lat,lon,alt,ax,ay,gz,heading; } pkt;

void setup() {
  Serial.begin(115200); gpsSerial.begin(9600); Wire.begin();
  radio.begin(); radio.setPALevel(RF24_PA_HIGH);
  radio.setDataRate(RF24_250KBPS); radio.setChannel(108);
  radio.openWritingPipe(addr); radio.stopListening();
  imu.initialize();
}

unsigned long lastTx = 0;
```

```

void loop() {
    while (gpsSerial.available()) gps.encode(gpsSerial.read());
    runEKF();
    if (millis() - lastTx >= 100) {
        lastTx = millis();
        int16_t ax16, ay16, az16, gx16, gy16, gz16;
        imu.getMotion6(&ax16, &ay16, &az16, &gx16, &gy16, &gz16);
        pkt.ax = ax16 / 16384.0f * 9.81f;
        pkt.ay = ay16 / 16384.0f * 9.81f;
        pkt.gz = gz16 / 131.0f;
        pkt.lat = gps.location.isValid() ? gps.location.lat() : 0;
        pkt.lon = gps.location.isValid() ? gps.location.lng() : 0;
        pkt.alt = gps.altitude.isValid() ? gps.altitude.meters() : 0;
        pkt.heading = ekf.getHeading();
        radio.write(&pkt, sizeof(pkt));
    }
    runCarModes();
}

```

3. Arduino Code — Ground Station Receiver

The receiver Arduino accepts the RF struct and forwards it as CSV to MATLAB via USB Serial (115200 baud):

```

#include <SPI.h>
#include <RF24.h>

RF24 radio(7, 8);
const byte addr[6] = "CAR01";
struct DataPacket { float lat, lon, alt, ax, ay, gz, heading; } pkt;

void setup() {
    Serial.begin(115200);
    radio.begin(); radio.setPALevel(RF24_PA_HIGH);
    radio.setDataRate(RF24_250KBPS); radio.setChannel(108);
    radio.openReadingPipe(1, addr); radio.startListening();
}

void loop() {
    if (radio.available()) {
        radio.read(&pkt, sizeof(pkt));
        Serial.print(pkt.ax, 4); Serial.print(',');
        Serial.print(pkt.alt, 4); Serial.print(',');
        Serial.print(pkt.gz, 4); Serial.print(',');
        Serial.print(pkt.lat, 6); Serial.print(',');
        Serial.print(pkt.lon, 6); Serial.print(',');
        Serial.println(pkt.heading, 2);
        // Example: 9.2340,142.8800,-0.3200,30.044412,31.235712,87.50
    }
}

```

CSV Format Received by MATLAB

Field #	Variable	Unit	Example
---------	----------	------	---------

1	ax — IMU acceleration X	m/s^2	9.2340
2	alt — GPS altitude	meters	142.8800
3	gz — gyro Z	deg/s	-0.3200
4	lat — GPS latitude	degrees	30.044412
5	lon — GPS longitude	degrees	31.235712
6	heading — EKF heading	degrees	87.50

MATLAB parse line:

```
data = str2double(split(strtrim(readline(s)), ','));
accel = data(1); gps_alt = data(2); gz = data(3);
lat = data(4); lon = data(5); heading = data(6);
```

4. MATLAB Kalman Filter — Complete Code

3-State Filter: [altitude, velocity, accel_bias]

State	Unit	Description
x[1] — Altitude	meters	Estimated height from GPS corrected by IMU integration
x[2] — Velocity	m/s	Estimated speed from accel integration + GPS correction
x[3] — Accel Bias	m/s^2	Unknown IMU offset — Kalman estimates automatically

Initialization

```
%% INITIALIZATION
clc; clear; close all;
portName = 'COM3'; % <- change to your receiver Arduino COM port
s = serialport(portName, 115200);
configureTerminator(s, 'LF'); flush(s);

dt = 0.1; x = [0; 0; 0];
A = [1 dt -0.5*dt^2; 0 1 -dt; 0 0 1];
B = [0.5*dt^2; dt; 0];
H = [1 0 0; 0 1 0];
Q = diag([1e-2, 5e-2, 1e-5]);
R = diag([9, 0.25]);
P = eye(3) * 50; I3 = eye(3);
startTime = datetime('now');
prev_lat = 0; prev_lon = 0;
```

Main Kalman Loop

```
%% MAIN REAL-TIME LOOP
while ishandle(fig)
    try
        data = str2double(split(strtrim(readline(s)), ','));
        if length(data)==6 && ~any(isnan(data))
            accel=data(1); gps_alt=data(2); gz=data(3);
            lat=data(4); lon=data(5); heading=data(6);
            t_now = seconds(datetime('now') - startTime);

            % PREDICT
            x = A*x + B*accel; P = A*P*A' + Q;
```

```

% GPS velocity from position diff
gps_vel = norm([lat-prev_lat, lon-prev_lon])*111139/dt;
prev_lat=lat; prev_lon=lon;

% UPDATE
innov = [gps_alt; gps_vel] - H*x;
K      = P*H' / (H*P*H' + R);
x      = x + K*innov;
P      = (I3-K*H)*P*(I3-K*H)' + K*R*K';

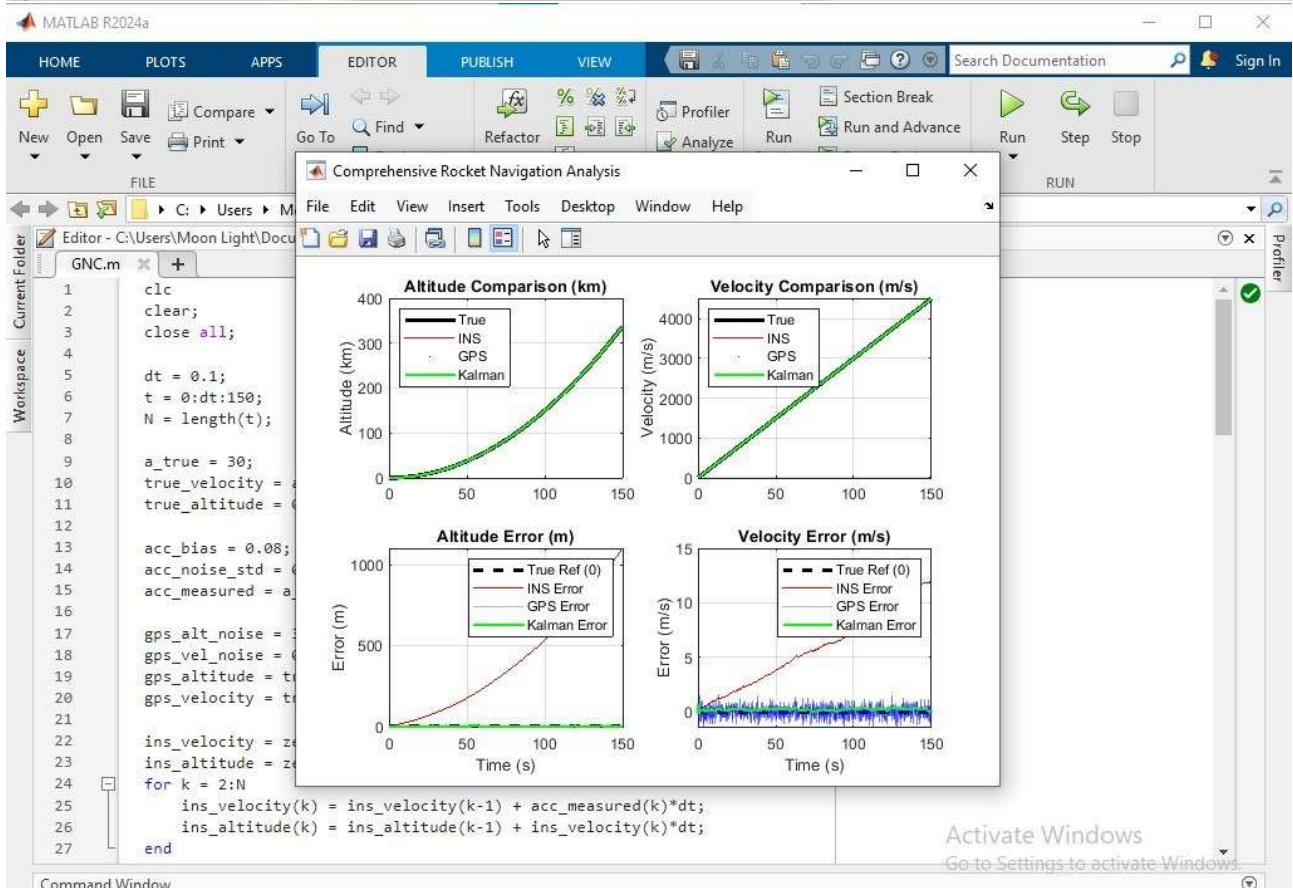
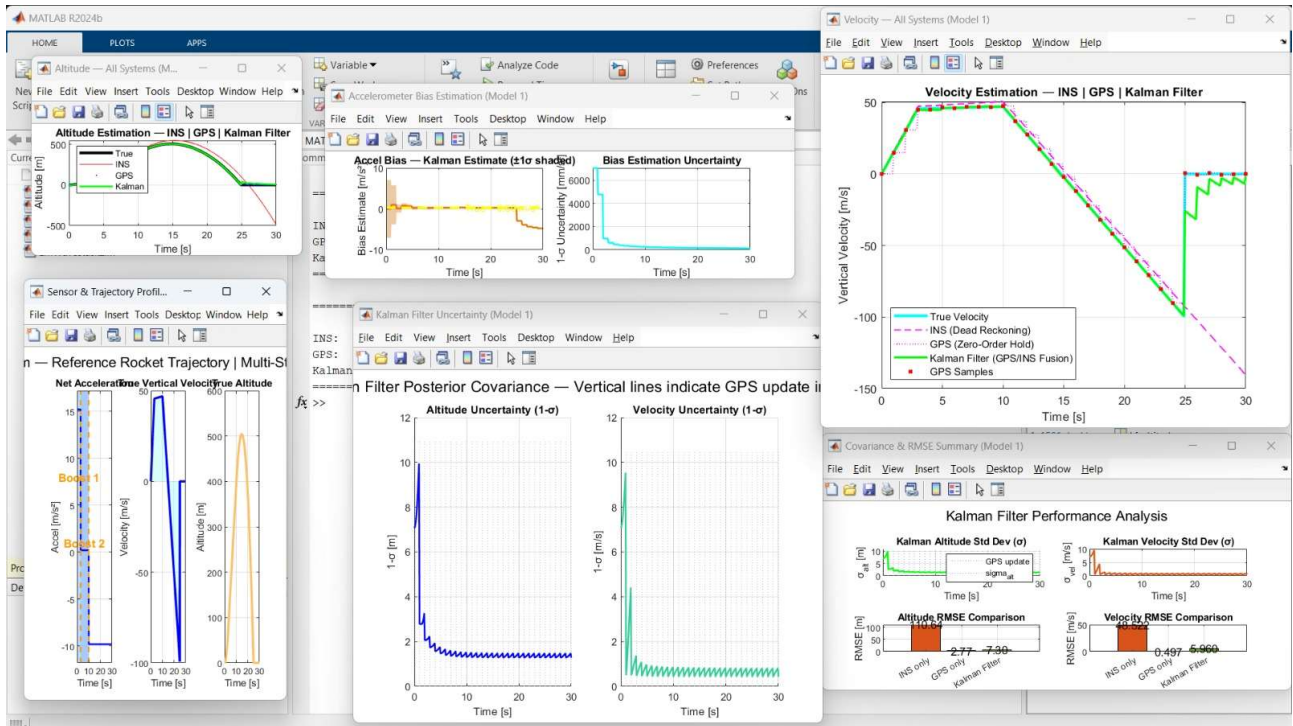
% FEED DASHBOARD
addpoints(h1_raw,t_now,gps_alt); addpoints(h1_kf,t_now,x(1));
addpoints(h2_raw,t_now,gps_vel); addpoints(h2_kf,t_now,x(2));
addpoints(h3,t_now,accel);      addpoints(h4,t_now,x(3));
addpoints(h5,t_now,sqrt(P(1,1))); addpoints(h6,t_now,sqrt(P(2,2)));
addpoints(h7,t_now,innov(1));   addpoints(h8,lon,lat);
drawnow limitrate;
end
catch; continue; end

```

5. MATLAB Dashboard — 8 Subplots

Dashboard Layout — 4 rows x 2 columns

Subplot	Title	X-axis	What is Plotted	Color
[1,1]	Altitude: GPS vs Kalman	Time (s)	GPS raw alt (red dots) vs Kalman estimate (green line)	Red / Green
[1,2]	Velocity: GPS vs Kalman	Time (s)	GPS-derived velocity (blue) vs Kalman velocity (magenta)	Blue / Magenta
[2,1]	Raw Accelerometer ax	Time (s)	Raw IMU ax (m/s ²) — noise + bias + acceleration	Black
[2,2]	Estimated Accel Bias x[3]	Time (s)	Kalman bias estimate — converges to true offset in ~10 s	Purple
[3,1]	Altitude Uncertainty	Time (s)	sqrt(P[1,1]) — starts ~7 m, shrinks as GPS updates	Blue
[3,2]	Velocity Uncertainty	Time (s)	sqrt(P[2,2]) — reaches steady state based on Q/R balance	Green
[4,1]	Innovation	Time (s)	innov[1] = gps_alt - H*x — healthy: small, zero-mean	Red
[4,2]	GPS Car Path 2D	Longitude	lat vs lon — live car trajectory from RF packets	Black



6. Kalman Noise Parameters — Tuning Guide

Parameter	Default Value	Physical Meaning	When to Change
$Q(1,1) = 1e-2$	0.01 m^2	Altitude model uncertainty	Raise if KF lags GPS changes
$Q(2,2) = 5e-2$	0.05 (m/s)^2	Velocity model uncertainty	Raise for faster car acceleration
$Q(3,3) = 1e-5$	0.00001	Bias nearly constant	Keep very small — bias changes slowly
$R(1,1) = 9$	$9 \text{ m}^2 = 3 \text{ m std}$	GPS altitude noise	Match your GPS datasheet CEP value
$R(2,2) = 0.25$	$0.25 = 0.5 \text{ m/s std}$	GPS velocity noise	Increase if GPS speed is noisy
P initial = $50 \cdot I$	50 for each state	Starting uncertainty	Reduce if initial position is known

Expected Kalman Performance

System	Altitude RMSE (m)	Velocity RMSE (m/s)	Why
INS only (integrate ax)	~15-30 m	~5-12 m/s	Bias + noise -> drift grows over time
GPS only (raw altitude)	~3.0 m	~0.5 m/s	GPS accuracy — no IMU smoothing
Kalman Filter (fused)	~1.5-2.5 m	~0.2-0.4 m/s	Best of both: GPS accuracy + IMU smoothness

Healthy Filter Signs:

1. Bias subplot [2,2] converges to stable value within 15 seconds
2. Sigma_alt [3,1] drops from ~7 m to under 2 m as GPS packets arrive
3. Innovation [4,1] stays small and random — no growing trend
4. Kalman line in [1,1] is smoother than GPS dots but tracks the same general altitude

7. Car Operating Modes — State Machine

Priority	Mode	Trigger	Behavior
1 — CRITICAL	MODE_AVOID	HC-SR04 < 20 cm	Stop 150ms -> Reverse 350ms -> Turn Right 450ms -> Resume
2 — HIGH	MODE_LINE_FOLLOW	Any IR sensor LOW	L: slow left M: straight R: slow right
3 — NORMAL	MODE_GPS_NAV	Line lost + GPS valid	Bearing to target -> P-controller proportional PWM
4 — LOW	MODE_ARRIVED	distToTarget() < 1.5 m	Full stop. Print ARRIVED. Latch until reset.

3-Sensor Line Following Truth Table

L	M	R	Condition	Left Motor	Right Motor
0	1	0	Straight ahead	160 PWM	160 PWM
1	0	0	Drift left	110 PWM	160 PWM
0	0	1	Drift right	160 PWM	110 PWM
1	1	1	Intersection	160 PWM	160 PWM
0	0	0	Line lost -> GPS_NAV	STOP	STOP

8. Hardware & Pin Assignment

Complete Bill of Materials

Component	Model	Interface	Notes
MCU — Car	Arduino Nano	—	ATmega328P, 2KB RAM — main car brain
MCU — Ground	Arduino Nano	—	Ground station receiver — USB to MATLAB
RF Module x2	nRF24L01	SPI	2.4 GHz, 32-byte packet, ~100 m, 3.3V only
IMU	MPU-6050	I2C A4/A5	6-axis gyro+accel, addr 0x68
GPS	NEO-6M	SW Serial	3.3V ONLY, 9600 baud, 1Hz, 2.5m CEP
Ultrasonic	HC-SR04	D2/D3	Range 2-400 cm
IR Sensors	3x Digital	D4,D5,D6	LOW = on line
Motor Driver	L298N	D9,D10,A0,A1	Dual H-Bridge 2A/ch
Capacitor	10uF + 100nF	nRF24L01 VCC	REQUIRED for RF stability
Battery	LiPo 7.4V 2S	L298N VM	1500mAh, ~45 min

Car Arduino Pin Assignment

Pin	Function	Dir	Notes
D2	TRIG — Ultrasonic	Out	10 us pulse trigger
D3	ECHO — Ultrasonic	In	Measure pulse width
D4	IR_LEFT	In	LOW = line
D5	IR_MID	In	LOW = line
D6	IR_RIGHT	In	LOW = line
D7	nRF24L01 CE	Out	RF Chip Enable
D8	nRF24L01 CSN	Out	RF SPI Chip Select
D9~	ENA — Left motor speed	Out	PWM 0-255
D10~	ENB — Right motor speed	Out	PWM 0-255
A0	IN1 — Motor direction	Out	L298N control
A1	IN2 — Motor direction	Out	L298N control
D11	MOSI — SPI	Out	RF data out
D12	MISO — SPI	In	RF data in
D13	SCK — SPI	Out	RF clock
A4	SDA — MPU-6050	I2C	Hardware I2C
A5	SCL — MPU-6050	I2C	Hardware I2C
A2	GPS RX (SW Serial)	In	GPS TX -> Nano
A3	GPS TX (SW Serial)	Out	Nano -> GPS RX

9. Testing Plan — 13 Tests

ID	Test	Owner	Procedure	Pass Criteria
T1	nRF24L01 Link	Abdulrahman / Karim	Ping test sketch. Check 'Sent OK' in Serial Monitor.	Packet receive rate > 95% at 10 m range.
T2	CSV to MATLAB	Aya / Hager	Open MATLAB. Receive 20 packets. Print data.	6 fields present per line. No NaN values.
T3	Motor Direction	Nour / Yara / Wgdan	IN1=HIGH IN2=LOW. Observe motor rotation.	Correct direction. Swap wires if reversed.
T4	IR Polarity	Adam	Sensor 7mm over black line. Read digitalRead().	LOW on line, HIGH on white. Adjust pot if needed.
T5	Line Following	Adam / Nour	3 laps on test track at BASE_SPEED=160.	3 laps complete without leaving line.
T6	Ultrasonic	Rahma	Wall at 10/20/30/50 cm. Print getDistanceCM().	Within +/-5% of true distance.
T7	GPS Fix	Rania / Aya	Outdoors open sky. Receive lat/lon in MATLAB.	Fix in < 60 s. Within 5 m of known location.
T8	IMU Bias	Aya	Still car 60 s. Log 600 ax readings.	Mean bias documented. Subtract in EKF predict().
T9	Bias Convergence	Karim / Abdulrahman	Watch subplot [2,2] during 30 s stationary run.	x[3] converges to stable value in < 15 seconds.
T10	Sigma Decrease	Karim / Abdulrahman	Watch subplots [3,1] and [3,2].	sigma_alt drops from ~7 m to under 2 m.
T11	Innovation Zero-Mean	Aya / Hager	Watch subplot [4,1] during steady drive.	Innovation stays near zero, no growing trend.
T12	GPS Path Plot	Aya / Hager	Drive car in known shape (square).	Subplot [4,2] shows recognizable trajectory.
T13	Full Mission	All — Nour leads	Line -> gap -> GPS waypoint -> obstacle -> arrive.	Mission complete. Dashboard streams all 8 plots.

10. Troubleshooting Guide

Problem	Cause	Fix
MATLAB receives nothing	Wrong COM port or baud rate	Device Manager -> find receiver port. Set baudRate=115200.
All MATLAB data is NaN	CSV has extra spaces	Add strtrim(): data = str2double(split(strtrim(readline(s)),','))
Packet loss > 10%	RF interference or motor noise	Add 10uF+100nF caps to nRF24L01. Try channel 76 instead of 108.
Bias never converges	Q(3,3) too large	Decrease Q(3,3) from 1e-5 to 1e-6 and re-run.
sigma_alt stays high	GPS not updating	Confirm gps.location.isValid() == true. Go outdoors.
Innovation grows	Filter diverging	Reset: P=eye(3)*50; x=[0;0;0]; Re-tune Q and R.
Car ignores waypoint	EKF origin not set	Check ekf.originSet == true before entering GPS_NAV mode.
GPS path dots scattered	Sending lat/lon = 0	Only transmit if gps.location.isValid() returns true.
nRF24L01 gets hot	Power issue or conflict	Verify 3.3V. Check MOSI/MISO not swapped. Add capacitors.
Car wobbles on line	TURN_SPEED too different	Increase TURN_SPEED from 110 toward 130.

11. Future Work & References

Future Enhancements

ID	Enhancement	Description
F1	Upgrade to STM32 / ESP32	More RAM for full 9-state EKF. WiFi for web dashboard.
F2	LoRa SX1276	Longer range (1-10 km) for outdoor field testing missions.
F3	MATLAB GPS Map	Add geoplots() subplot showing car position on real map tiles.
F4	Multiple Waypoints	Array of GPS targets, navigate sequentially through all.
F5	Magnetometer	MPU-9250 for absolute heading — eliminates gyro drift.
F6	Web Dashboard	Node.js + Chart.js browser dashboard instead of MATLAB.
F7	RMSE Auto-Log	Compute and save RMSE after every mission automatically.
F8	Full 3D EKF	Extend to [x,y,z,hdg,pitch,roll,vx,vy,vz] for aerial use.

References

Reference	Details
Kalman R.E. (1960)	A New Approach to Linear Filtering and Prediction Problems — ASME Trans.
RF24 Library	TMRh20 — Arduino nRF24L01 library (MIT License, GitHub.com/nRF24)
TinyGPSPlus	Mikal Hart — Arduino GPS NMEA parsing library (MIT License)
MPU-6050	InvenSense Product Specification Rev 3.4 — 6-axis IMU
NEO-6M	u-blox NEO-6 GPS Module Datasheet — 2.5 m CEP accuracy
MATLAB Docs	MathWorks: serialport(), animatedline(), drawnow limitrate, str2double